Functional Requirements PRD

Customer, inventory, warehouse, tasks, scheduling and fulfillment behavior

1. Functional Requirements

This section defines the behavior QuickFlow must provide from order creation through fulfillment completion.

2. Customer Order Journey

```
Login → Browse Products → Cart → Checkout → Place Order → Track Order
```

- Customer can select products and quantities.
- Customer can choose delivery speed such as Standard, Express or Same Day.
- Delivery choice contributes to order priority/deadline handling.
- Customer receives an order identifier after successful creation.
- Customer can view current fulfillment state and progress.

3. Order Intake

React sends the order to the Node.js/Express API. The backend validates the request, persists the order and invokes the C++ engine for fulfillment decisions.

Input	Required Concept
Customer	Customer identity / authorization
Items	Product IDs and quantities
Destination	Delivery location
Delivery type	Standard / Express / Same Day
Priority / deadline	Scheduling inputs

4. Inventory Check

The engine checks every relevant warehouse to determine whether the required items and quantities are available. Warehouses unable to fulfill the order become ineligible candidates.

```
Delhi: Laptop ✓, Mouse ✓ Mumbai: Laptop ✓, Mouse ✓ Hyderabad: Laptop ✗ Kolkata: Laptop ✓, Mouse ✗
```

5. Warehouse Selection

If multiple warehouses are eligible, QuickFlow computes a fulfillment score using the project's proposed inputs: distance, inventory availability, current workload, processing time and shipping cost. Lower score is better.

Functional Requirements PRD

Customer, inventory, warehouse, tasks, scheduling and fulfillment behavior

6. Inventory Reservation

After warehouse selection, inventory is reserved for the order. This operation is concurrency-sensitive because several requests may attempt to reserve the same remaining stock at the same time.

Example: if one laptop remains and customers A, B and C submit orders concurrently, only one reservation may succeed for that final unit. The engine should protect the critical section using locking / atomic mechanisms.

7. Order → Task Decomposition

```
Order ■■ Pick items ■■ Pack order ■■ Quality check ■■ Ship order
```

Tasks carry taskId, orderId, type, priority, deadline, status and assignedWorker. Picking work can be represented as multiple independent tasks, while downstream stages remain dependent on completion of upstream work.

8. Task Dependencies

```
PICK → PACK → QUALITY CHECK → SHIP
```

A completed picking stage unlocks packing; completed packing unlocks quality check; a successful quality check unlocks shipping. This dependency model prevents invalid execution order.

9. Scheduling Requirements

Strategy	Purpose
FIFO	Process work in arrival order.
Priority	Prefer higher-value / urgent work.
Deadline	Prefer work with the most pressing deadline.
Hybrid	Combine priority and deadline signals.

The scheduler must be replaceable without rewriting the rest of the fulfillment pipeline. The project notes propose the Strategy Pattern for this behavior.

10. Dynamic Priority

Priority may increase as an order approaches its deadline. For example, a normal order with priority 3 can be promoted toward priority 9 when only a short time remains before a same-day deadline.

Functional Requirements PRD

Customer, inventory, warehouse, tasks, scheduling and fulfillment behavior

11. Worker Assignment

The engine maintains worker status, department/skill, current task, completed tasks, efficiency and availability. The scheduler assigns eligible tasks to available workers.

```
Task Queue ↓ Best available worker ↓ ASSIGNED → IN_PROGRESS → COMPLETED
```

12. Parallel Execution

Independent tasks may execute concurrently. A thread pool avoids creating a new operating-system thread for every task. Fixed worker threads take tasks from a synchronized queue.

13. Failure Handling

```
Task → Worker Failure → FAILED → Retry Queue → Another Worker ↓ After retry limit ↓ MANUAL REVIEW
```

The proposed policy is to retry a failed task up to three attempts before escalating it for manual review. The implementation should preserve task state and event history during each attempt.

14. Order State Machine

```
CREATED → CONFIRMED → ALLOCATED → PICKING → PACKING → QUALITY_CHECK → SHIPPED → DELIVERED
```

Only valid state transitions are allowed. For example, DELIVERED → PICKING must be rejected. The State Pattern can encapsulate state-specific transition rules.

15. Quality & Shipping

- Quality check validates product, quantity, damage condition and order identity.
- A failed QC result sends the order for reprocessing rather than shipping.
- After QC success, the shipping task transfers fulfillment to the delivery network.
- The final customer-visible state is DELIVERED.